```
class Actor
  soliloquy: ->
    'To be or not to be...'
  stunt: ->
    'My arm! Call an ambulance.'

scene = ->
  imaStarr = new Actor
  imaStarr.soliloquy()
  imaStarr.stunt()
  'Scene completed'
```

An example Actor class.

The scene depends on the actor.

A test for this scene asserts that it returns the value `'Scene completed'`:

```
fact 'The scene is completed', ->
  assert.equals scene(), 'Scene completed'
```

The scene itself obtains the actor, so a test for this scene has no way to replace the actor with a double. If instead the scene is provided with the actor, then replacing the actor with a double becomes possible:

```
scene = (actor) ->
  actor.soliloquy()
  actor.stunt()
  'Scene completed'
```

Now you can replace the actor with a double by passing it in as an argument. The double will need to be convincing, though. It's no good having a double that doesn't know how to `soliloquy` because the scene won't work if it doesn't. So how do you create a test double that works?

Frameworks

If you use a test framework, it will likely provide some standard ways to create test doubles. But as with assertions, you don't need a testing framework to learn how to use doubles. In some languages it's very difficult to create a double without a framework, but in CoffeeScript it's quite easy. This is because objects are like an all-night café—always open.

Creating a double from scratch